

OBJECT ORIENTED PROGRAMMING UNIT - V

**CS305PC: OBJECT ORIENTED PROGRAMMING
THROUGH JAVA**



CONCEPTS OF APPLETS

- Applets are small applications that are accessed on an Internet server, transported over the Internet, automatically installed, and run as part of a Web document.
- After an applet arrives on the client, it has limited access to resources, so that it can produce an arbitrary multimedia user interface and run complex computations without introducing the risk of viruses or breaching data integrity.

CONCEPTS OF APPLETS

- applets – Java program that runs within a Java-enabled browser, invoked through a “applet” reference on a web page, dynamically downloaded to the client computer.

```
import java.applet.*;
import java.awt.*;
public class SimpleApplet extends Applet {
    public void paint(Graphics g) {
        g.drawString("A Simple Applet", 20, 20);
    }
}
```

CONCEPTS OF APPLET

- There are two ways to run an applet:
 1. Executing the applet within a Java-compatible Web browser, such as NetscapeNavigator.
 2. Using an applet viewer, such as the standard JDK tool, appletviewer.
- An appletviewer executes your applet in a window. This is generally the fastest and easiest way to test an applet.
- To execute an applet in a Web browser, you need to write a short HTML text file that contains the appropriate APPLET tag.
 - `<applet code="SimpleApplet" width=200 height=60>`
 - `</applet>`

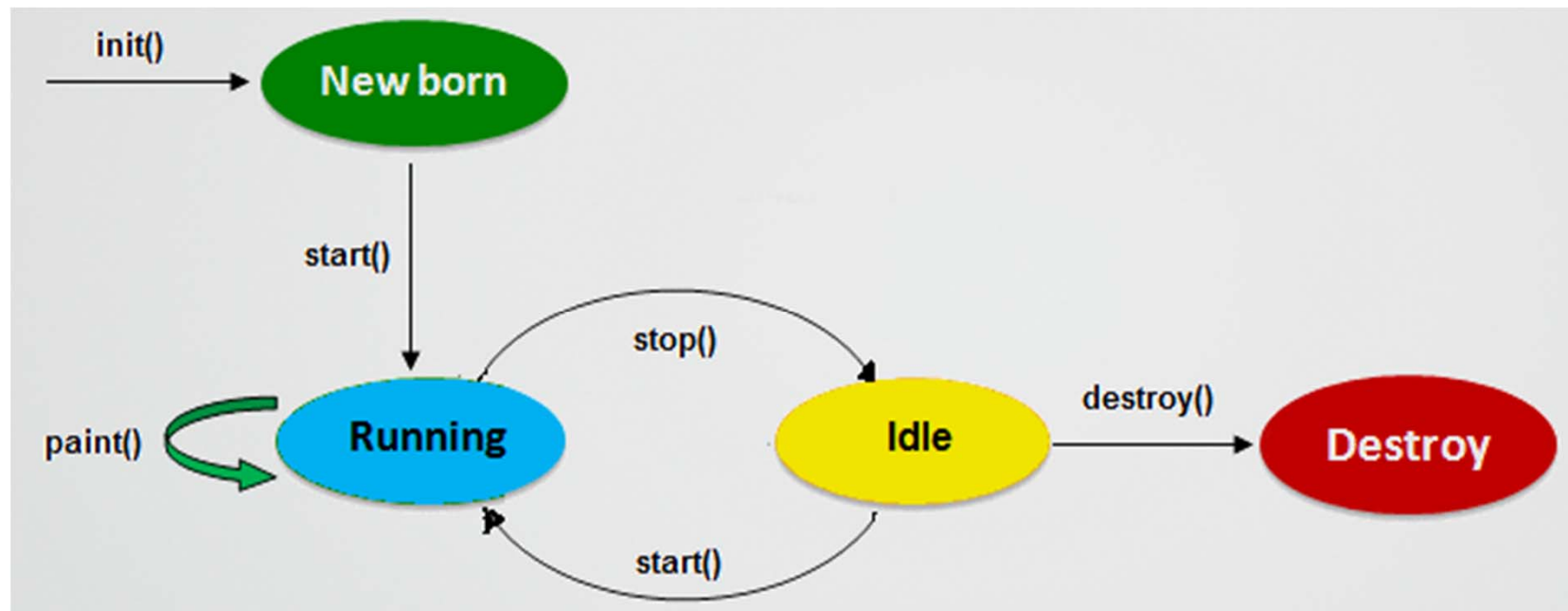
DIFFERENCES BETWEEN APPLETS AND APPLICATIONS

- Java can be used to create two types of programs: applications and applets.
- An application is a program that runs on your computer, under the operating system of that Computer(i.e an application created by Java is more or less like one created using C or C++).
- When used to create applications, Java is not much different from any other computer language.
- An applet is an application designed to be transmitted over the Internet and executed by a Java-compatible Web browser.
- An applet is actually a tiny Java program, dynamically downloaded across the network, just like an image, sound file, or video clip.

DIFFERENCES BETWEEN APPLETS AND APPLICATIONS

- The important difference is that an applet is an intelligent program, not just an animation or media file(i.e an applet is a program that can react to user input and dynamically change not just run the same animation or sound over and over.
- Applications require main method to execute.
- Applets do not require main method.
- Java's console input is quite limited
- Applets are graphical and window-based.

LIFE CYCLE OF AN APPLLET



LIFE CYCLE OF AN APPLET

- Applets life cycle includes the following methods
 - init() | start() | paint()
 - stop() | destroy()
- When an applet begins, the AWT calls the following methods, in this sequence:
 - init() | start() | paint()
- When an applet is terminated, the following sequence of method calls takes place:
 - stop() | destroy()

LIFE CYCLE OF AN APPLET

- The **init()** method is the first method to be called. This is where you should initialize variables. This method is called only once during the run time of your applet.
- The **start()** method is called after init(). It is also called to restart an applet after it has been stopped. Whereas init() is called once—the first time an applet is loaded—start() is called each time an applet's HTML document is displayed onscreen. So, if a user leaves a web page and comes back, the applet resumes execution at start().

LIFE CYCLE OF AN APPLLET

- The **paint()** method is called each time applet's output must be redrawn. **paint()** is also called when the applet begins execution. Whatever the cause, whenever the applet must redraw its output, **paint()** is called. The **paint()** method has one parameter of type **Graphics**. This parameter will contain the graphics context, which describes the graphics environment in which the applet is running. This context is used whenever output to the applet is required.
- The **stop()** method is called when a web browser leaves the HTML document containing the applet—when it goes to another page, for example. When **stop()** is called, the applet is probably running. Applet uses **stop()** to suspend threads that don't need to run when the applet is not visible.

LIFE CYCLE OF AN APPLLET

- To restart `start( )` is called if the user returns to the page.
- The **`destroy( )`** method is called when the environment determines that your applet needs to be removed completely from memory. The `stop( )` method is always called before `destroy( )`.

TYPES OF APPLETS

- Applets are two types
 - 1.Simple applets
 - 2.JApplets
- Simple applets can be created by extending Applet class
- JApplets can be created by extending JApplet class of javax.swing.JApplet package

CREATING APPLETS

- Applets are created by extending the Applet class.

```
import java.awt.*;
import java.applet.*;
/*<applet code="AppletSkel" width=300 height=100></applet> */
public class AppletSkel extends Applet {
    public void init() { initialization }
    public void start() { start or resume execution }
    public void stop() { suspends execution }
    public void destroy() { perform shutdown activities }
    public void paint(Graphics g) { redisplay contents of window }
}
```

PASSING PARAMETERS TO APPLETS

- APPLET tag in HTML allows you to pass parameters to applet.
- To retrieve a parameter, use the `getParameter( )` method. It returns the value of the specified parameter in the form of a String object.

```
// Use Parameters
import java.awt.*;
import java.applet.*;
/*
<applet code="ParamDemo" width=300 height=80>
<param name=fontName value=Courier>
<param name=fontSize value=14>
<param name=leading value=2>
<param name=accountEnabled value=true>
</applet>
*/
```

PASSING PARAMETERS TO APPLETS

```
import java.applet.Applet;

public class ParamDemo extends Applet{
    String fontName;
    int fontSize;
    float leading;
    boolean active;
    // Initialize the string to be displayed.
    public void start() {
        String param;
        fontName = getParameter("fontName");
        if(fontName == null)
            fontName = "Not Found";
        param = getParameter("fontSize");
        try {
            if(param != null) // if not found
                fontSize = Integer.parseInt(param);
            else
                fontSize = 0;
        } catch(NumberFormatException e) {
            fontSize = -1;
        }
    }
}
```


PASSING PARAMETERS TO APPLETS

```
    param = getParameter("leading");
    try {
        if(param != null) // if not found
            leading = Float.valueOf(param).floatValue();
        else
            leading = 0;
    } catch(NumberFormatException e) {
        leading = -1;
    }
    param = getParameter("accountEnabled");
    if(param != null)
        active = Boolean.valueOf(param).booleanValue();
}
// Display parameters.
public void paint(Graphics g) {
    g.drawString("Font name: " + fontName, 0, 10);
    g.drawString("Font size: " + fontSize, 0, 26);
    g.drawString("Leading: " + leading, 0, 42);
    g.drawString("Account Active: " + active, 0, 58);
}
}
```

INTRODUCTION TO SWINGS

- Swing is a set of classes that provides more powerful and flexible components than are possible with the AWT.
- In addition to the familiar components, such as buttons, check boxes, and labels, Swing supplies several exciting additions, including tabbed panes, scroll panes, trees, and tables.
- Even familiar components such as buttons have more capabilities in Swing.

INTRODUCTION TO SWINGS

- For example, a button may have both an image and a text string associated with it. Also, the image can be changed as the state of the button changes.
- Unlike AWT components, Swing components are not implemented by platform-specific code.
- Instead, they are written entirely in Java and, therefore, are platform-independent.
- The term lightweight is used to describe such elements.

SWING COMPONENTS

- The Swing components are defined in javax.swing
 - **AbstractButton**: Abstract superclass for Swing buttons.
 - **ButtonGroup**: Encapsulates a mutually exclusive set of buttons.
 - **ImageIcon**: Encapsulates an icon.
 - **JApplet**: The Swing version of Applet.
 - **JButton**: The Swing push button class.
 - **JCheckBox**: The Swing check box class.
 - **JComboBox** : Encapsulates a combo box (an combination of a drop-down list and text field).

SWING COMPONENTS

- JLabel: The Swing version of a label.
- JRadioButton: The Swing version of a radio button.
- JScrollPane: Encapsulates a scrollable window.
- JTabbedPane: Encapsulates a tabbed window.
- JTable: Encapsulates a table-based control.
- JTextField: The Swing version of a text field.
- JTree: Encapsulates a tree-based control.

LIMITATIONS OF AWT

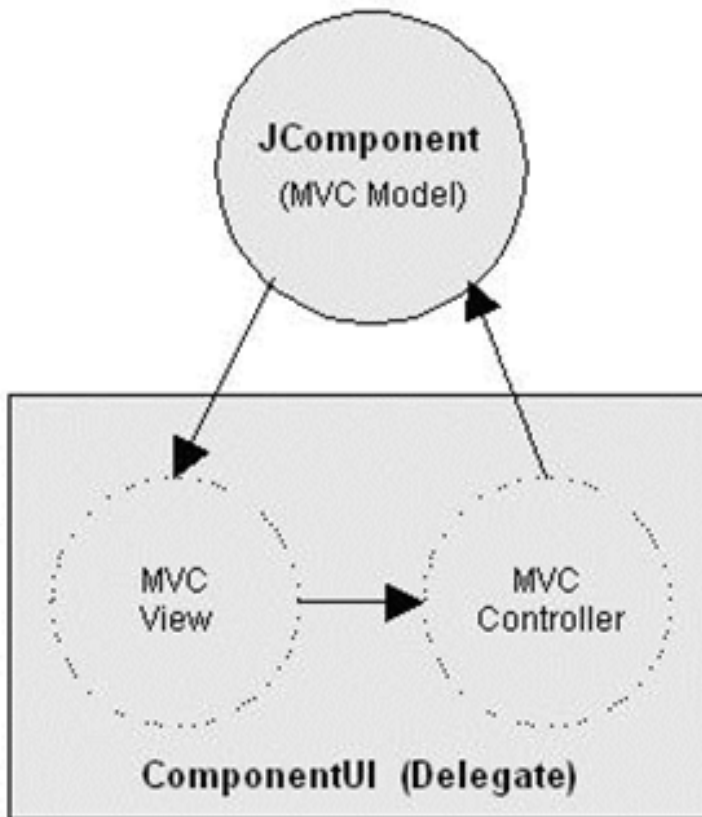
- AWT supports limited number of GUI components.
- AWT components are heavy weight components.
- AWT components are developed by using platform specific code.
- AWT components behaves differently in different operating systems.
- AWT component is converted by the native code of the operating system.

LIMITATIONS OF AWT

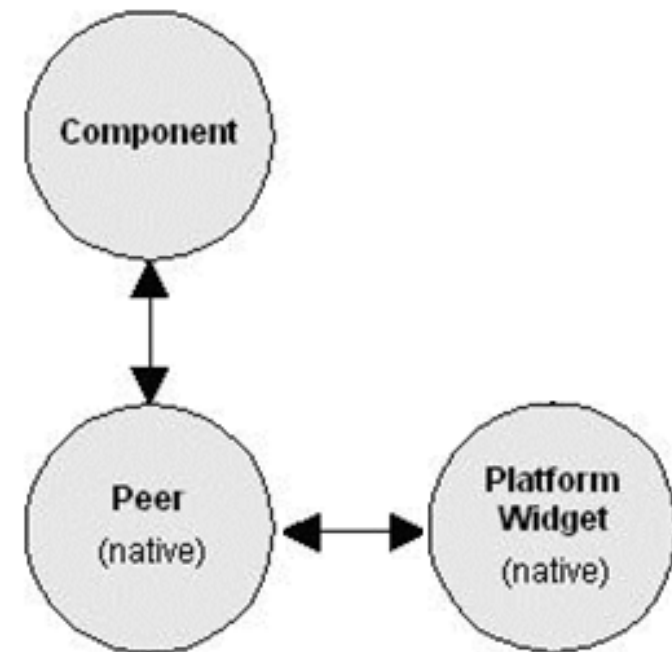
- **Lowest Common Denominator**
 - If not available natively on one Java platform, not available on any Java platform
- **Simple Component Set**
- **Components Peer-Based**
 - Platform controls component appearance
 - Inconsistencies in implementations
 - Interfacing to native platform error-prone

LOOK & FEEL (SWING | AWT)

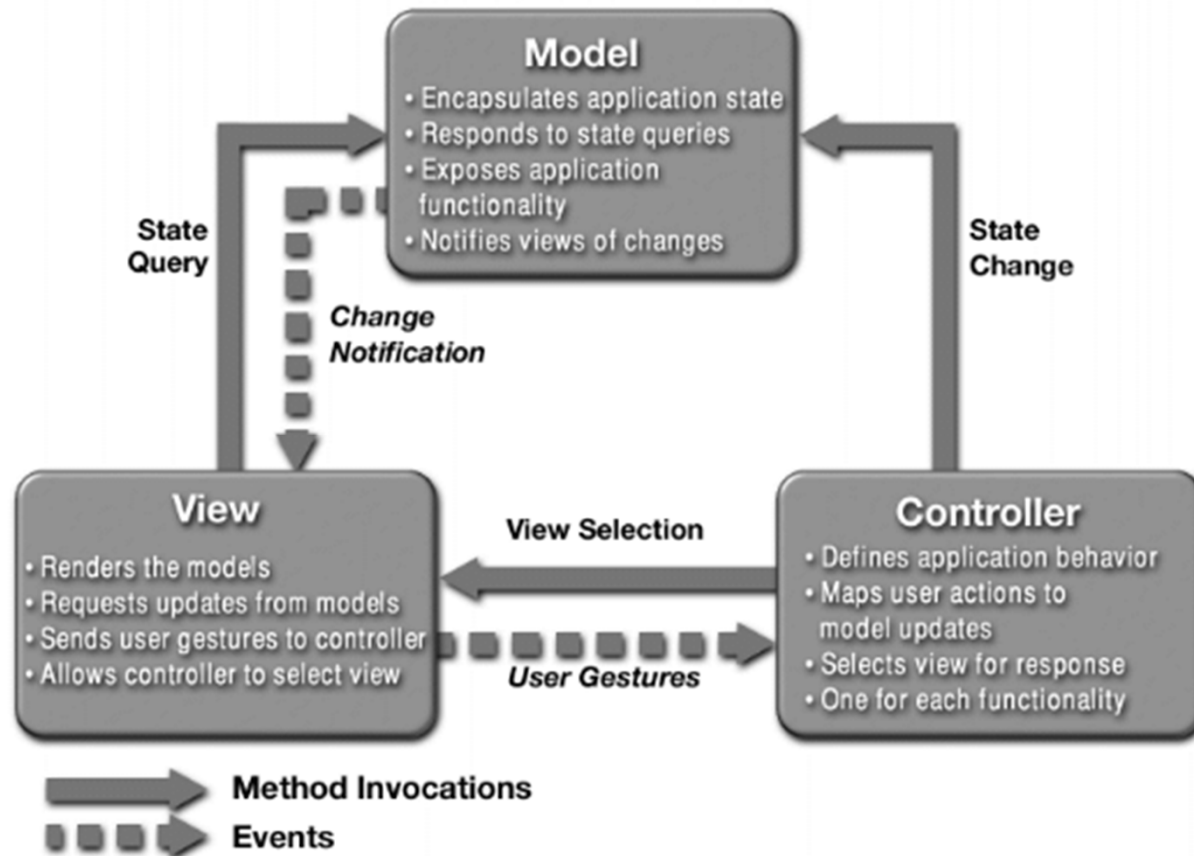
Swing Look & Feel



AWT Look & Feel



MVC ARCHITECTURE



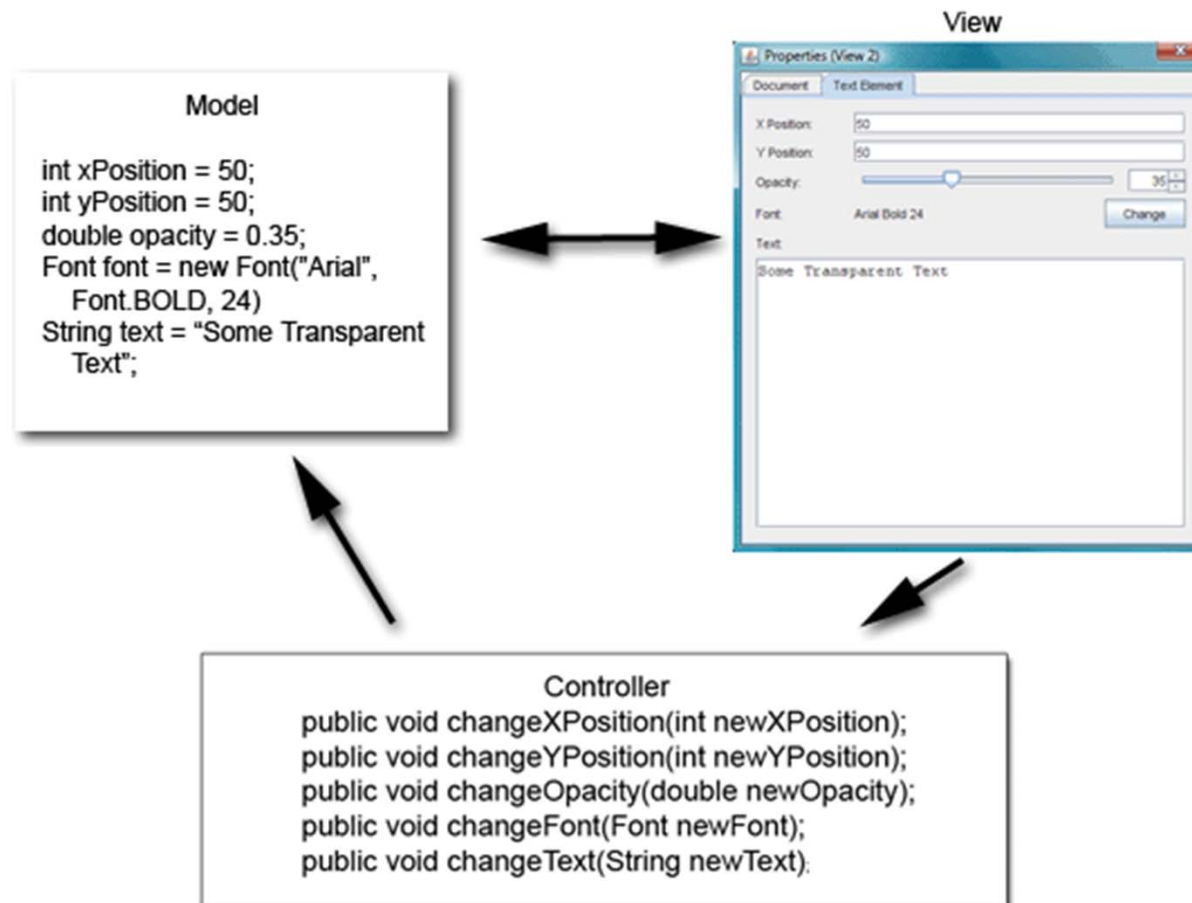
MODEL, VIEW

- **Model** - The model represents data and the rules that govern access to and updates of this data. In enterprise software, a model often serves as a software approximation of a real-world process.
- **View** - The view renders the contents of a model. It specifies exactly how the model data should be presented. If the model data changes, the view must update its presentation as needed. This can be achieved by using a push model, in which the view registers itself with the model for change notifications, or a pull model, in which the view is responsible for calling the model when it needs to retrieve the most current data.

CONTROLLER

- **Controller** - The controller translates the user's interactions with the view into actions that the model will perform. In a stand-alone GUI client, user interactions could be button clicks or menu selections, whereas in an enterprise web application, they appear as GET and POST HTTP requests. Depending on the context, a controller may also select a new view
- -- for example, a web page of results -- to present back to the user.

INTERACTION B/W MVC COMPONENTS



COMPONENTS

- **Container**

- **JComponent**

- AbstractButton
 - JButton
 - JMenuItem
 - JCheckBoxMenuItem
 - JMenu
 - JRadioButtonMenuItem
 - JToggleButton
 - JCheckBox
 - JRadioButton

- **JComponent**

- JComboBox
 - JLabel
 - JList
 - JMenuBar
 - JPanel
 - JPopupMenu
 - JScrollBar
 - JScrollPane

- **JComponent**

- JTextComponent
 - JTextArea
 - JTextField
 - JPasswordField
 - JTextPane
 - JHTMLPane

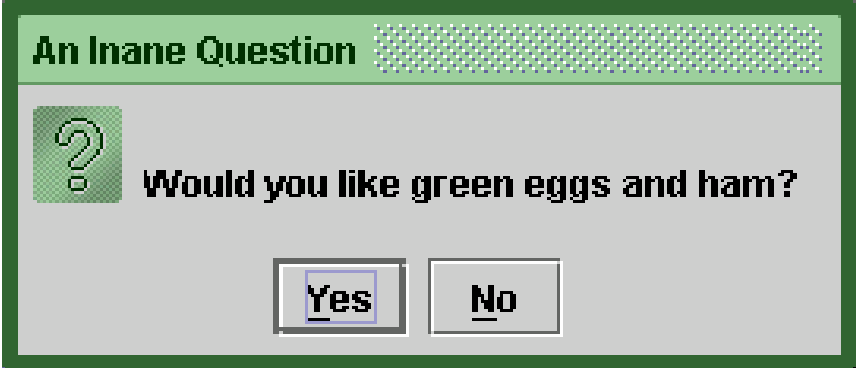
CONTAINERS

- Top-Level Containers
- The components at the top of any Swing containment hierarchy

Top-Level Containers



Applet



Dialog



Frame

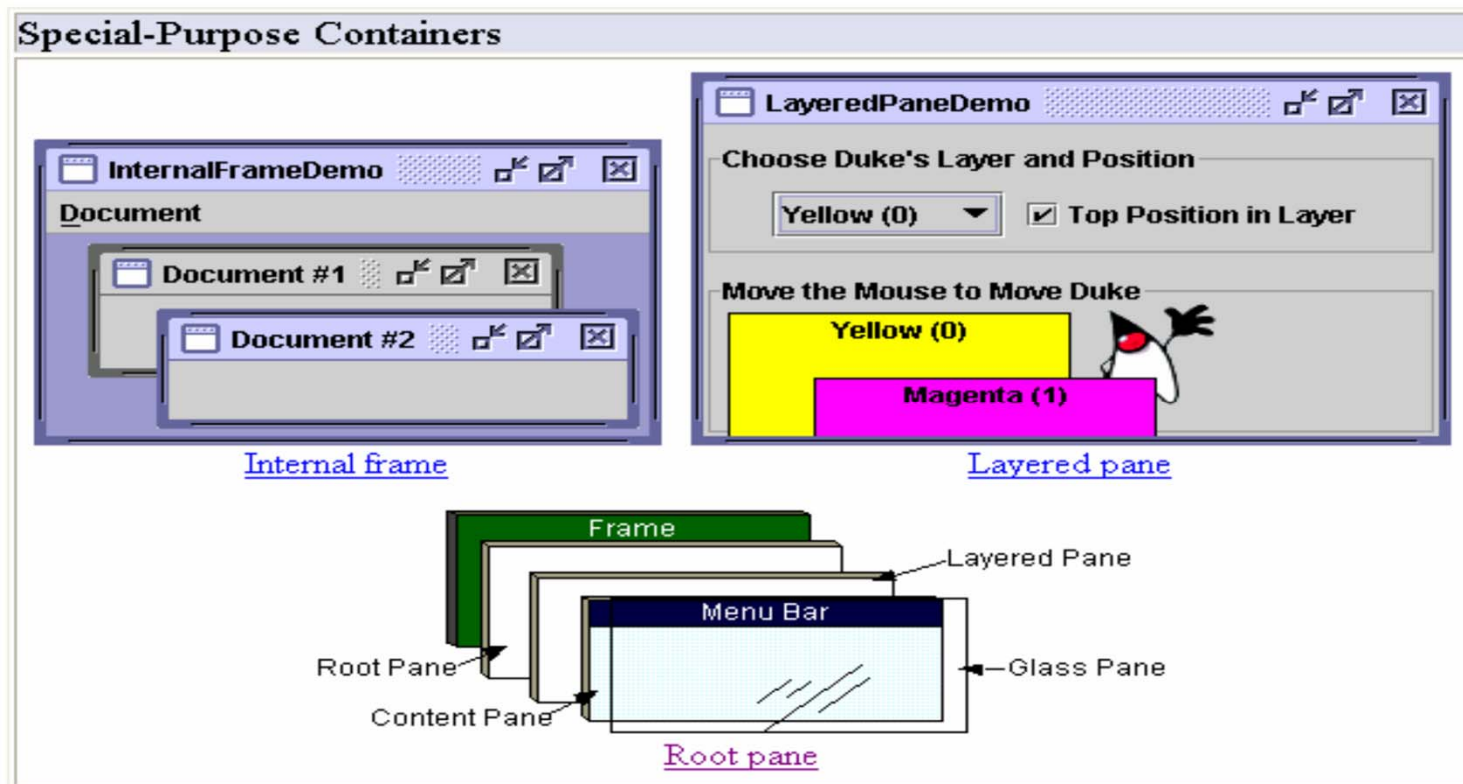
GENERAL PURPOSE CONTAINERS

- Intermediate containers that can be used under many different circumstances.



SPECIAL PURPOSE CONTAINER

- Intermediate containers that play specific roles in the UI.



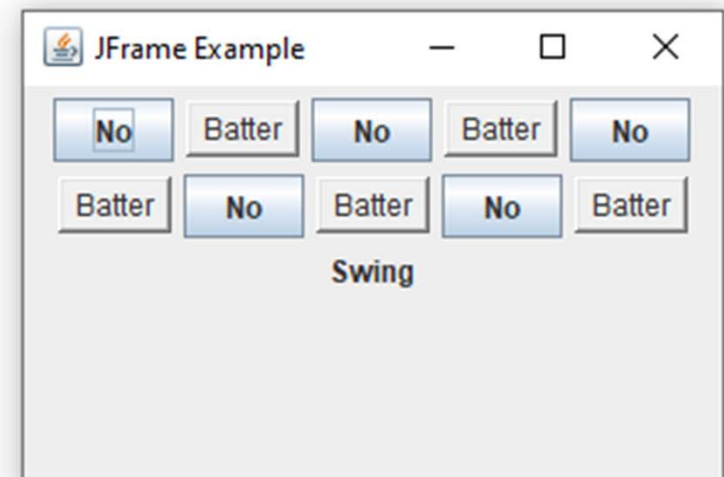
EXPLORING SWING - JAPPLET

- If using Swing components in an applet, subclass JApplet, not Applet
 - JApplet is a subclass of Applet
 - Sets up special internal component event handling, among other things
 - Can have a JMenuBar
 - Default LayoutManager is BorderLayout

JFRAME

```
import javax.swing.*;
import java.awt.*;

public class FrameTest {
    public static void main (String args[]) {
        JFrame f = new JFrame ("JFrame Example");
        Container c = f.getContentPane();
        c.setLayout (new FlowLayout());
        for (int i = 0; i < 5; i++) {
            c.add (new JButton ("No"));
            c.add (new Button ("Batter"));
        }
        c.add (new JLabel ("Swing"));
        f.setSize (300, 200);
        f.show();
    }
}
```



JCOMPONENT

- JComponent supports the following components.

JComboBox	JTextComponent
JLabel	JTextArea
JList	JTextField
JMenuBar	JPasswordField
JPanel	JTextPane
JPopupMenu	JHTMLPane
JScrollBar	
JScrollPane	

ICONS AND LABELS

- In Swing, icons are encapsulated by the ImageIcon class, which paints an icon from an image.
- constructors are:
 - ImageIcon(String filename)
 - ImageIcon(URL url)
- The ImageIcon class implements the Icon interface that declares the methods
 - int getIconHeight()
 - int getIconWidth()
 - void paintIcon(Component comp,Graphics g,int x, int y)

ICONS AND LABELS

- Swing labels are instances of the JLabel class, which extends JComponent. It can display text and/or an icon.
- Constructors are:
 - JLabel(Icon i) | Label(String s) | JLabel(String s, Icon i, int align)
- Here, s and i are the text and icon used for the label. The align argument is either LEFT, RIGHT, or CENTER. These constants are defined in the SwingConstants interface,
- Methods are:
 - Icon getIcon() | String getText() | void setIcon(Icon i)
 - void setText(String s)
 - Here, i and s are the icon and text, respectively.

TEXT FIELDS

- The Swing text field is encapsulated by the `JTextField` class, which extends `JComponent`.
- It provides functionality that is common to Swing text components.
- One of its subclasses is `JTextArea`, which allows you to edit one line of text.
- Constructors are:
 - `JTextField( )` | `JTextField(int cols)` | `JTextField(String s, int cols)`
 - `JTextField(String s)`
- Here, `s` is the string to be presented, and `cols` is the number of columns in the text field.

BUTTONS

- Swing buttons provide features that are not found in the Button class defined by the AWT.
- Swing buttons are subclasses of the AbstractButton class, which extends JComponent.
- AbstractButton contains many methods that allow you to control the behavior of buttons, check boxes, and radio buttons.
- Methods are:
 - `void setDisabledIcon(Icon di) | void setPressedIcon(Icon pi)`
 - `void setSelectedIcon(Icon si) | void setRolloverIcon(Icon ri)`

BUTTONS CONTD...

- Here, di, pi, si, and ri are the icons to be used for these different conditions.
- The text associated with a button can be read and written via the following methods:
 - `String getText( )`
 - `void setText(String s)`
- Here, s is the text to be associated with the button.

JBUTTON

- The JButton class provides the functionality of a push button.
- JButton allows an icon, a string, or both to be associated with the push button.
- Some of its constructors are :
 - JButton(Icon i)
 - JButton(String s)
 - JButton(String s, Icon i)
- Here, s and i are the string and icon used for the button.

CHECK BOXES

- The `JCheckBox` class, which provides the functionality of a check box, is a concrete implementation of `AbstractButton`.
- Some of its constructors are shown here:
 - `JCheckBox(Icon i) | JCheckBox(Icon i, boolean state)`
 - `JCheckBox(String s) | JCheckBox(String s, boolean state)`
 - `JCheckBox(String s, Icon i) | JCheckBox(String s, Icon i, boolean state)`
- Here, `i` is the icon for the button. The text is specified by `s`. If state is true, the check box is initially selected. Otherwise, it is not.
- The state of the check box can be changed via the following method:
 - `void setSelected(boolean state)`
 - Here, state is true if the check box should be checked.

COMBO BOXES

- Swing provides a combo box (a combination of a text field and a drop-down list) through the JComboBox class, which extends JComponent.
- A combo box normally displays one entry. However, it can also display a drop-down list that allows a user to select a different entry. You can also type your selection into the text field.
- Two of JComboBox's constructors are :
 - JComboBox() | JComboBox(Vector v)
 - Here, v is a vector that initializes the combo box.
- Items are added to the list of choices via the addItem() method, whose signature is:
 - void addItem(Object obj)
 - Here, obj is the object to be added to the combo box.

RADIO BUTTONS

- Radio buttons are supported by the `JRadioButton` class, which is a concrete implementation of `AbstractButton`.
- Some of its constructors are :
 - `JRadioButton(Icon i) | JRadioButton(Icon i, boolean state)`
 - `JRadioButton(String s) | JRadioButton(String s, boolean state)`
 - `JRadioButton(String s, Icon i) | JRadioButton(String s, Icon i, boolean state)`
- Here, `i` is the icon for the button. The text is specified by `s`. If `state` is `true`, the button is initially selected. Otherwise, it is not.
- Elements are then added to the button group via the following method:
 - `void add(AbstractButton ab)`
- Here, `ab` is a reference to the button to be added to the group.

TABBED PANES

- A tabbed pane is a component that appears as a group of folders in a file cabinet.
- Each folder has a title. When a user selects a folder, its contents become visible. Only one of the folders may be selected at a time.
- Tabbed panes are commonly used for setting configuration options. These are encapsulated by the `JTabbedPane` class, which extends `JComponent`. We will use its default constructor. Tabs are defined via the following method:
 - `void addTab(String str, Component comp)`
 - Here, `str` is the title for the tab, and `comp` is the component that should be added to the tab. Typically, a `JPanel` or a subclass of it is added.

TABBED PANES

- The general procedure to use a tabbed pane in an applet is outlined here:
 1. Create a JTabbedPane object.
 2. Call addTab() to add a tab to the pane. (The arguments to this method define the title of the tab and the component it contains.)
 3. Repeat step 2 for each tab.
 4. Add the tabbed pane to the content pane of the applet.

SCROLL PANES

- A scroll pane is a component that presents a rectangular area in which a component may be viewed. Horizontal and/or vertical scroll bars may be provided if necessary.
- Scroll panes are implemented in Swing by the JScrollPane class, which extends JComponent. Some of its constructors are :
 - JScrollPane(Component comp)
 - JScrollPane(int vsb, int hsb)
 - JScrollPane(Component comp, int vsb, int hsb)
- Here, comp is the component to be added to the scroll pane. vsb and hsb are int constants that define when vertical and horizontal scroll bars for this scroll pane are shown.

SCROLL PANES

- These constants are defined by the ScrollPaneConstants interface.
 - HORIZONTAL_SCROLLBAR_ALWAYS
 - HORIZONTAL_SCROLLBAR_AS_NEEDED
 - VERTICAL_SCROLLBAR_ALWAYS
 - VERTICAL_SCROLLBAR_AS_NEEDED
- Here are the steps to follow to use a scroll pane in an applet:
 - 1. Create a JComponent object.
 - 2. Create a JScrollPane object. (The arguments to the constructor specify the component and the policies for vertical and horizontal scroll bars.)
 - 3. Add the scroll pane to the content pane of the applet.

TREES

- Data Model - `TreeModel`
 - default: `DefaultTreeModel`
 - `getChild`, `getChildCount`, `getIndexOfChild`, `getRoot`, `isLeaf`
- Selection Model - `TreeSelectionModel`
- View - `TreeCellRenderer`
 - `getTreeCellRendererComponent`
- Node - `DefaultMutableTreeNode`

TABLES

- A table is a component that displays rows and columns of data. You can drag the cursor on column boundaries to resize columns. You can also drag a column to a new position.
- Tables are implemented by the `JTable` class, which extends `JComponent`.
- One of its constructors is :
 - `JTable(Object data[ ][ ], Object colHeads[ ])`
 - Here, `data` is a two-dimensional array of the information to be presented, and `colHeads` is a one-dimensional array with the column headings.

TABLES

- Here are the steps for using a table in an applet:
 - 1. Create a `JTable` object.
 - 2. Create a `JScrollPane` object. (The arguments to the constructor specify the table and the policies for vertical and horizontal scroll bars.)
 - 3. Add the table to the scroll pane.
 - 4. Add the scroll pane to the content pane of the applet.